



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 08**

**NOMBRE COMPLETO:** Jiménez Elizalde Josué

**N° de Cuenta:** 320334489

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 06

**SEMESTRE** 2026-1

**FECHA DE ENTREGA LÍMITE:** 23/10/2025

**CALIFICACIÓN:** \_\_\_\_\_

## REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para esta practica se pidieron los siguientes ejercicios.

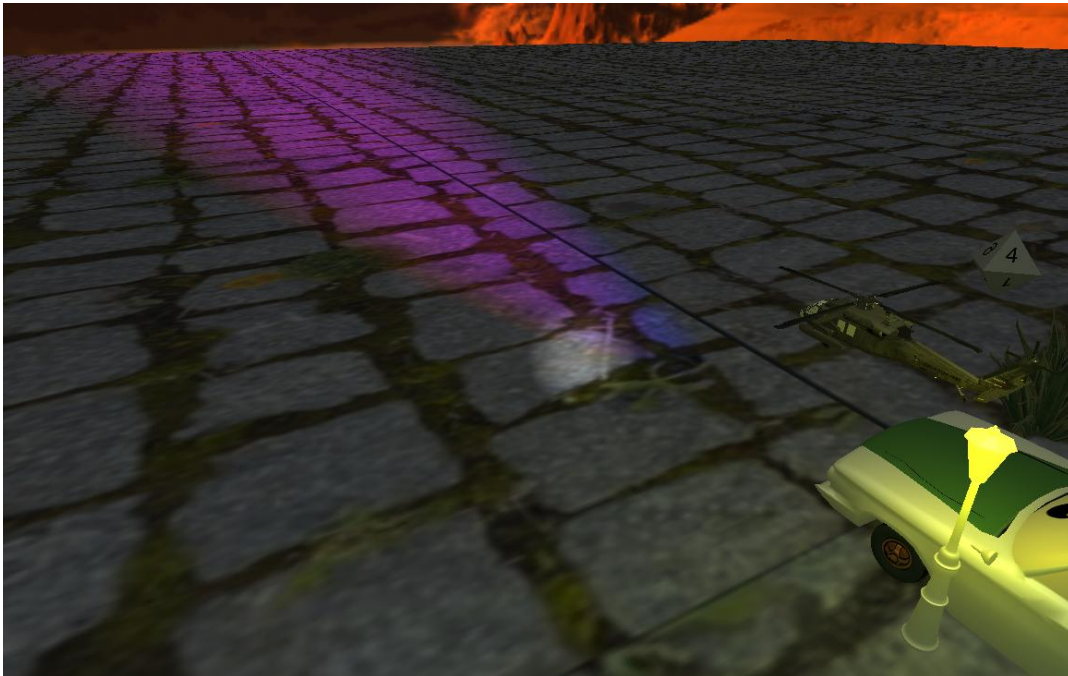
1. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Se coloco el foco en la parte del cofre haciendo que suba y baje con el cofre haciendo lo siguiente

```
//Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(-2.43f, 2.43f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()),
glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
CofreLuzPos = glm::vec4(-2.0, 2.0f, -1.0f, 1.0f);
CofreLuz = glm::vec3(model * CofreLuzPos);
spotLights[2].SetPos(glm::vec3(CofreLuz));
spotLights[2].SetFlash(
    CofreLuz,
    glm::vec3(-cos(glm::radians(mainWindow.getarticulacion2()))),
    sin(glm::radians(mainWindow.getarticulacion2()))), 0.0f)
);
```

Con este código y con setFlash se tiene el movimiento circular con cosenos y senos para su movimiento en ese sentido. Todo lo demás se explicó en anteriores prácticas.

Con esto se tiene el siguiente resultado, como primer foto se tiene sin que haya movimiento



Como siguiente se presentara el movimiento.



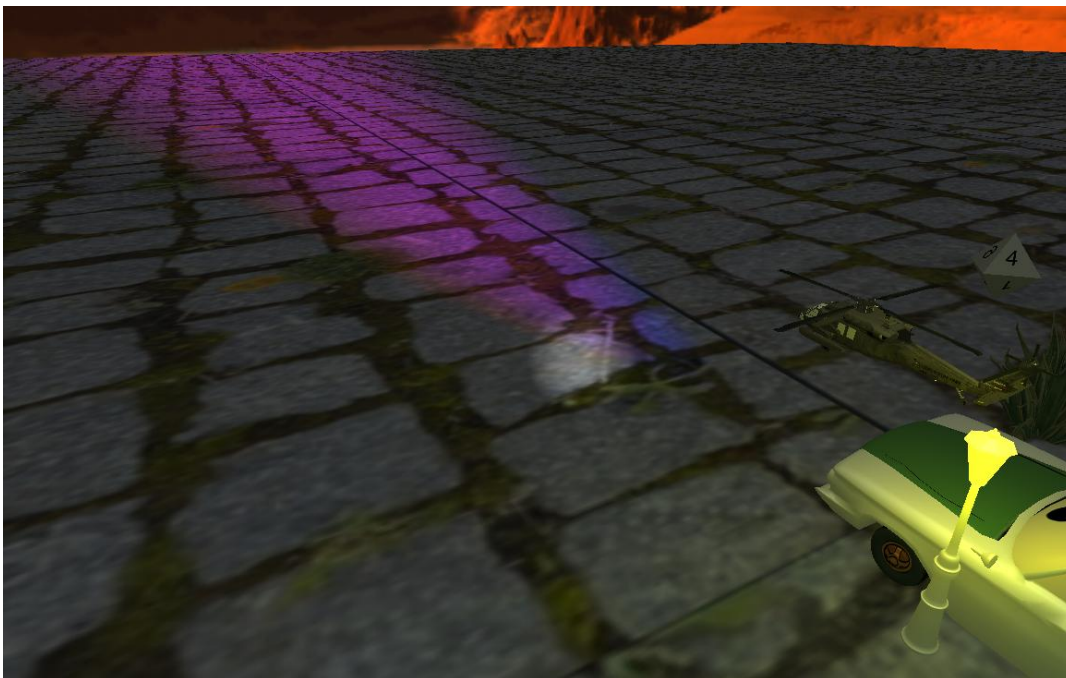
En el helicóptero se ve el color que se mueve.

2. Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa/positiva) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X opuesta) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagaran de acuerdo con alguna bandera asignada por ustedes.

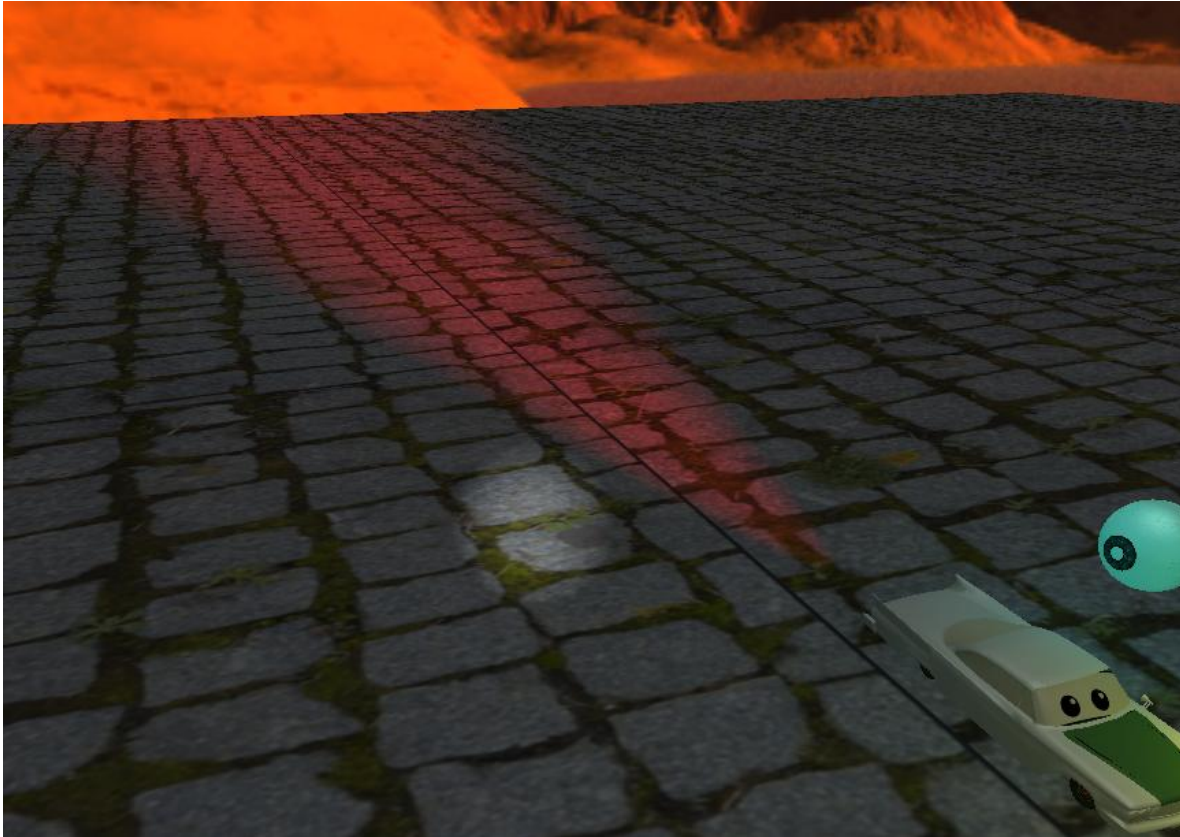
Con estos se tiene la validación para las luces cuando se avance con la tecla Y y U para el movimiento y solo agregando la siguiente validación.

```
if (mainWindow.getLuces() < 0.5f)
{
    spotLights[1] = auxSpot[1];
    spotLights[1].SetPos(glm::vec3(10.6f + cochePos.x, 0.7f + cochePos.y,
3.0f + cochePos.z));
}
else
{
    spotLights[1] = auxSpot[0];
    spotLights[1].SetPos(glm::vec3(-2.6f + cochePos.x, 0.7f + cochePos.y,
3.0f + cochePos.z));
}
```

Con esto renderiza una u otra, cuando va para enfrente se pone una luz azul y cuando va para atrás se pone roja.



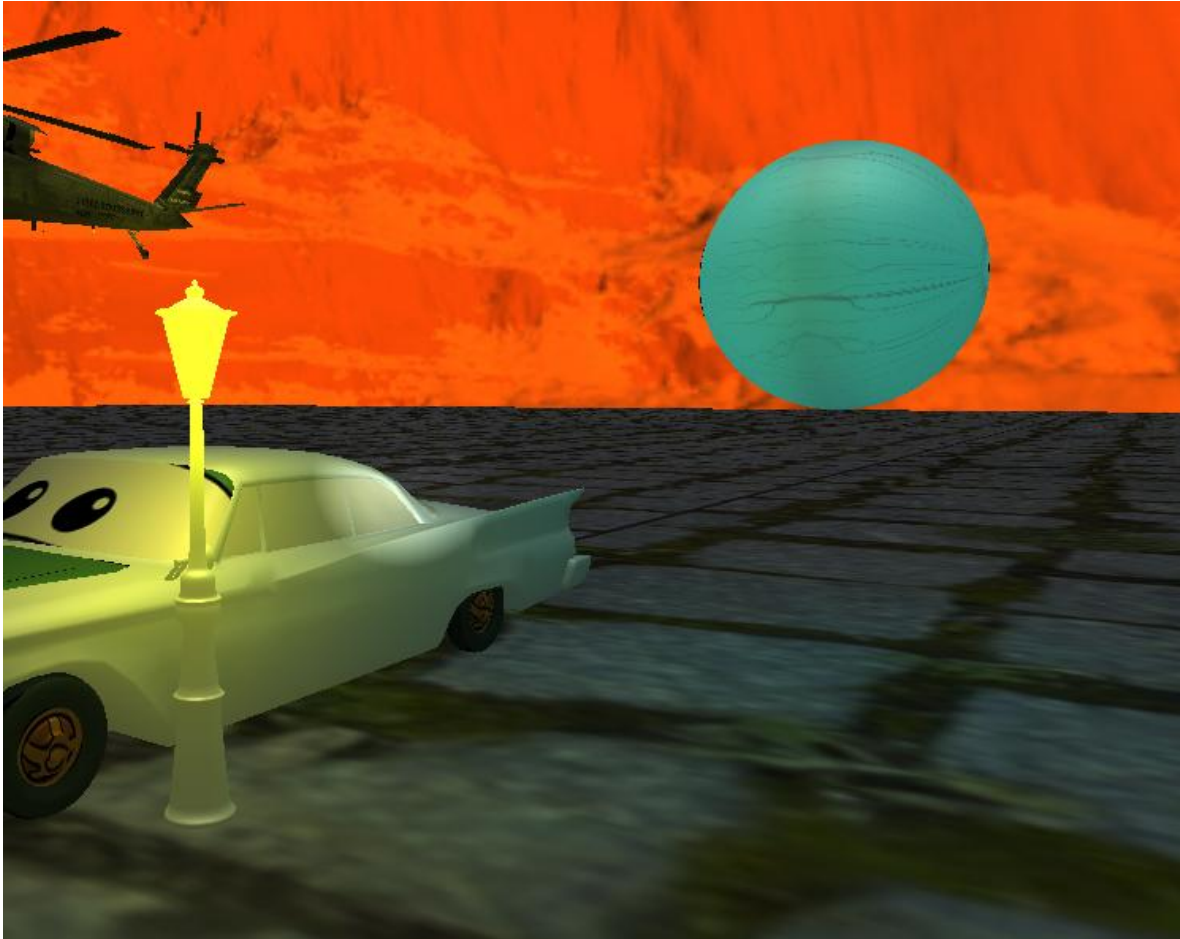
Para atrás:



Así se demuestra los valores.

3.- Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo ( la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual).

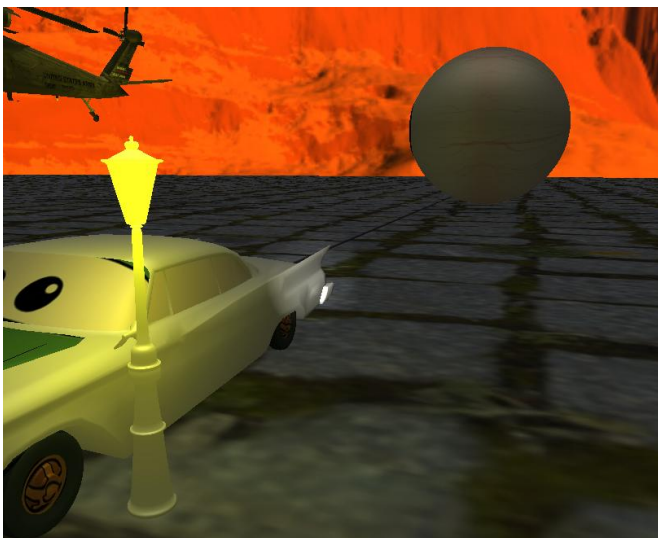
Para este ejercicio se agrego un modelo como se explicaba en anteriores prácticas.



En este caso es el ojo y se presentara la forma de apagar alguna de ellas.

Ya se mostro cuando están encendidas las dos.

Cuando solo esta encendida la lampara.



Cuando solo este encendido el ojo



Cuando ninguno de los dos está encendido.



Simplemente se utilizo la siguiente validación en código.

```
if (mainWindow.getPrendido() > 0.5f && mainWindow.getPrendido2() > 0.5f)
{
    printf("%s\n", "Hola1");
    shaderList[0].SetPointLights(pointLights, pointLightCount);
}
else if (mainWindow.getPrendido() > 0.5f && mainWindow.getPrendido2() <= 0.5f)
{
    printf("%s\n", "Hola2");
    shaderList[0].SetPointLights(pointLights, 1);
}
else if (mainWindow.getPrendido() <= 0.5f && mainWindow.getPrendido2() > 0.5f)
{
    printf("%s\n", "Hola3");
    shaderList[0].SetPointLights(pointLights + 1, 1);
}
else
{
    printf("%s\n", "Hola4");
    shaderList[0].SetPointLights(pointLights, 0);
}
```

Con esta validación se prendía o apagaba las luces de los modelos una por una. Los printf se tienen para ver si entraba en el evento.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No se tuvo problemas a la hora de elaborar la práctica.

3.- Conclusión:

Se cuenta con los conocimientos suficientes para manejar la iluminación en proyectos futuros, con esta práctica y la anterior se refuerza conocimientos de manejo de luces, además de los valores y como imprimir mas de la cantidad máxima.

Bibliografía en formato APA

Ing. José Roque RG (septiembre 2025) Laboratorio de Computación Gráfica e

Interacción Humano Computadora. Clase Facultad de Ingeniería UNAM